线性DP讲题顺序：

ABCFGHQWZ

AT_dp_a Frog 1

【题目翻译】

[Frog 1 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]

【解题思路】

这是一道典型的线性DP题

状态描述：用 $dp[i]$ 表示跳到第 i 块石头所需的最小费用。

状态转移方程：$dp[i] = min(dp[i] + cost(i, i-1), dp[i] + cost(i, i-2))$

初值：$dp[1] = 0, dp[2] = cost(1, 2)$

注意：大致的思路，可以产生各种细节不同的代码，只要能自圆其说，初值和状态转移方程配合好就行，以下面两段程序为例，初值、转移方程略有不同。

【参考程序一】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
int a[maxn],f[maxn];
int n;
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    f[2]=abs(a[2]-a[1]);
    for (int i=3;i<=n;i++)
        f[i]=min(f[i-2]+abs(a[i]-a[i-2]),f[i-1]+abs(a[i]-a[i-1]));
    printf("%d\n",f[n]);
}
```

【参考程序二】

```
#include<bits/stdc++.h>
using namespace std;
const int inf=2e9+50;
int n,a[101000],f[101000];
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d",&a[i]);
        f[i]=inf;

    }
    f[1]=0;
    for (int i=2;i<=n;i++){
        if (i-2>0) f[i]=min(f[i],f[i-2]+abs(a[i]-a[i-2]));
        if (i-1>0) f[i]=min(f[i],f[i-1]+abs(a[i]-a[i-1]));
    }
    printf("%d\n",f[n]);
}
```

AT_dp_b Frog2

【题目翻译】

[Frog 2 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]

【解题思路】

还是一道线性DP，在Frog1的基础上，增加了青蛙的跳跃方案。

状态：$dp[i]$ 表示青蛙跳到石头 i 时使用的力气的最小值。

状态转移：$dp[i]=min(dp[j]+abs(a[i]-a[j])),(i-k<=j<=i-1)$

初值：$dp[1]=0,$ 其他值设置成最大值

【参考程序一】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
const int inf=1e9;
int n,k,a[maxn],f[maxn];
int main(){
    scanf("%d%d",&n,&k);
    for (int i=1;i<=n;i++) {
        scanf("%d",&a[i]);
        f[i]=inf;
    }
    f[1]=0;
    for (int i=1;i<=n;i++)
        for (int j=i-k;j<=i-1;j++){
            if (j>=1) f[i]=min(f[i],f[j]+abs(a[i]-a[j]));
        }
    printf("%d\n",f[n]);

}
```

【参考程序二】

```
#include<bits/stdc++.h>
using namespace std;
const int inf=2e9;
int f[210000],a[210000];
int n,k;
int main(){

    scanf("%d%d",&n,&k);
    for (int i=1;i<=n;i++){
        scanf("%d",&a[i]);
        f[i]=inf;
    }
    f[1]=0;
    for (int i=2;i<=n;i++){
        for (int j=1;j<=k;j++)
            if (i-j>0) f[i]=min(f[i],f[i-j]+abs(a[i]-a[i-j]));
    }
    printf("%d\n",f[n]);
}
```

AT_dp_c Vacation

【题目翻译】

[Vacation - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://luogu.com.cn/problem/AT_dp_c)

【解题思路】

线性DP题，但是需要增加状态所需要描述的信息，因为仅仅知道某一天，状态转移没法正常进行。

状态描述：$f[i][j]$ 表示第i天玩的是j项目所能产生的最大幸福值。

状态转移方程：

$f[i][0] = max(f[i-1][1], f[i-1][2]) + a[i][0]$

$f[i][1] = max(f[i-1][0], f[i-1][2]) + a[i][1]$

$f[i][2] = max(f[i-1][0], f[i-1][1]) + a[i][2]$

初值：

$f[0][0] = 0, \text{其他设置成最小值}$

【参考程序一】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
int a[maxn][4],f[maxn][4],n;
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        for (int j=1;j<=3;j++)
            scanf("%d",&a[i][j]);
    }
    for (int i=1;i<=n;i++){
        for (int j=1;j<=3;j++)
            for (int k=1;k<=3;k++)
                if (j!=k) f[i][j]=max(f[i][j],f[i-1][k]+a[i][j]);
    }
    printf("%d\n",max(max(f[n][1],f[n][2]),f[n][3]));
}
```

【参考程序二】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
int n,a[maxn][4],f[maxn][5];
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d%d%d",&a[i][1],&a[i][2],&a[i][3]);
    }
    for (int i=1;i<=n;i++){
        for (int j=1;j<=3;j++){
            for (int k=1;k<=3;k++)
                if (j!=k) f[i][j]=max(f[i][j],f[i-1][k]+a[i][k]);
        }
    }
    printf("%d\n",max(max(f[n][1],f[n][2]),f[n][3]));
}
```

AT_dp_f LCS

【题目翻译】

[LCS - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://www.luogu.com.cn/problem/AT_dp_f)

【解题思路】

线性DP

令$X=<x_1,x_2,...,x_m>$和$Y=<y_1,y_2,...,y_n>$为两个序列，$Z=<z_1,z_2,...,z_k>$为X和Y的任意LCS。则：

如果$x_m=y_n$,则$z_k=x_m=y_n$且z_{k-1}是x_{m-1}和y_{n-1}的一个LCS

如果$x_m\neq y_n$,那么$z_k\neq x_m$,意识着z是x_{m-1}和y的一个LCS

如果$x_m\neq y_n$,那么$z_k\neq y_n$,意味着z是x和y_{n-1}的一个LCS

从上述的结论可以看出，两个序列的LCS问题包含两个序列的LCS，因此，LCS 问题具有最优子结构性质。在设计递归算法时，不难看出递归算法具有子问题重叠的性质。 总之，可以证明 LCS 是所有方法中最快的一种。

状态：$dp[i][j]$ 第一个序列的前i个，第二个序列的前j个，能够形成的公共子序列的最大长度。

转移方程：

$$dp[i][j]=\begin{cases}0\text{（}i==0,\ or,\ j==0) \\ dp[i-1][j-1]+1(i,j>0,and,x_i==y_j) \\ max(dp[i][j-1],dp[i-1][j])(i,j>0,and,x_i\neq y_j)\end{cases}$$

初值：上述方程也写了，$dp[0][0]=0$,其他都设置成最小值

这道题还需要输出具体的子序列

LCS 的本质就是一个填表的过程，从 $dp[0][0]$ 开始，一直到 $dp[n-1][m-1]$ 结束，此题我们还要同时考虑输出路径，所以为了下文表达方便，我们不妨设：

1. 在对应字符相等的时候，用 ↖ 标记。
2. 在 *p*1≥*p*2 的时候，用↑标记。
3. 在 *p*1<*p*2 的时候，用←标记

LCS 的路径输出

若想得到路径，则再遍历一次*d***p* 数组就好了，从最后一个位置开始往前遍历：

如果箭头是 ↖，则代表这个字符是 LCS 的一员，存下来后 `i--`，`j--`

如果箭头是 ←，则代表这个字符不是LCS的一员，`j--`

如果箭头是↑，也代表这个字符不是LCS的一员，`i--`

如此直到 `i = 0` 或者 `j = 0` 时停止，存下来的部分即为 LCS 的路径。

以如下为例，填表理解下LCS

```
ABCB
BDCA
```

【参考程序一】

```
#include<bits/stdc++.h>
using namespace std;
char a[5000],b[5000];
int f[5000][5000],L[5000][5000],R[5000][5000];
int n,m;
void print_ans(int x,int y){
    //printf("%d %d @@@\n",x,y);
    if (x==0&&y==0){
        return;
    }
    print_ans(L[x][y],R[x][y]);
    if (x>L[x][y]&&y>R[x][y]&&a[x]==b[y])
        printf("%c",a[x]);
}
int main(){
    scanf("%s",a+1);
    scanf("%s",b+1);
    n=strlen(a+1);
    m=strlen(b+1);
    for (int i=1;i<=n;i++)
        for (int j=1;j<=m;j++){
            //f[i][j]=max(f[i-1][j],f[i][j-1]);
            if (f[i][j]<f[i-1][j]){
                f[i][j]=f[i-1][j];
                L[i][j]=i-1;
                R[i][j]=j;
            }
            if (f[i][j]<f[i][j-1]){
                f[i][j]=f[i][j-1];
                L[i][j]=i;
                R[i][j]=j-1;
            }
            if (a[i]==b[j]){
                if (f[i][j]<f[i-1][j-1]+1){
                    f[i][j]=f[i-1][j-1]+1;
                    L[i][j]=i-1;
                    R[i][j]=j-1;
                }
            }
                ;
        }
    //printf("%d %d %d\n",f[1][1],L[1][1],R[1][1]);
    print_ans(n,m);
    //printf("%d\n",f[n][m]);
}
```

【参考程序二】

```
#include<bits/stdc++.h>
using namespace std;
const int inf=-1e9;
char a[4000],b[4000];
int f[3010][3010],g[3010][3010][3];
int n,m;
void pri_ans(int x,int y){
```

```
    //printf("%d %d\n",x,y);
    if (x==0 || y==0) return;
    int tmp1=g[x][y][1];
    int tmp2=g[x][y][2];
    pri_ans(tmp1,tmp2);
    if (x>tmp1&&y>tmp2) {
        //printf("%d %d %d %d\n",x,y,tmp1,tmp2);
        printf("%c",a[x]);
    }
}
int main(){
    scanf("%s",a+1);
    scanf("%s",b+1);
    n=strlen(a+1);
    m=strlen(b+1);
    for (int i=0;i<=n;i++)
        for (int j=0;j<=m;j++)
            f[i][j]=inf;
    f[0][0]=0;
    for (int i=0;i<=n;i++)
        for (int j=0;j<=m;j++)
        {
            if (i==0&&j==0) continue;
            if (a[i]==b[j])
                if (f[i][j]<f[i-1][j-1]+1){
                    f[i][j]=f[i-1][j-1]+1;
                    g[i][j][1]=i-1;
                    g[i][j][2]=j-1;
                }
            if (f[i][j]<f[i-1][j]){
                f[i][j]=f[i-1][j];
                g[i][j][1]=i-1;
                g[i][j][2]=j;

            }
            if (f[i][j]<f[i][j-1]){
                f[i][j]=f[i][j-1];
                g[i][j][1]=i;
                g[i][j][2]=j-1;
            }
        }
//  printf("%d\n",f[n][m]);
    //printf("%d %d %d\n",f[2][4],g[2][4][1],g[2][4][2]);
    pri_ans(n,m);

}
```

AT_dp_g Longest Path

【题目翻译】

[Longest Path - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://www.luogu.com.cn/problem/AT_dp_g)

【解题思路】

有向无环图（DAG）模型上的DP（使用记忆化搜索实现）

状态：$dp[i]$ 表示从i出发能够形成的最长路

状态转移：$dp_i = max(0, max_{i,j \in E}(dp_j + 1))$

初值：递归终点，没有相邻结点，只有自身一个长度

【参考程序】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
int n,m,ans,x,y,d[maxn];
vector<int>a[maxn];
int dfs(int x){
    //printf("%d\n",x);
    if (d[x]>0) return d[x];
    d[x]=1;
    //for (auto i:a[x])
    vector<int>::iterator it;
    for (it=a[x].begin();it!=a[x].end();it++)
        d[x]=max(d[x],dfs(*it)+1);
    return d[x];
}
int main(){
    scanf("%d%d",&n,&m);
    for (int i=1;i<=m;i++){
        scanf("%d%d",&x,&y);
        a[x].push_back(y);
    }
    int ans=-1;
    for (int i=1;i<=n;i++){
        ans=max(ans,dfs(i));
        //printf("%d\n",d[i]);
    }

    printf("%d\n",ans-1);

}
```

DAG上的DP专练（矩形嵌套1431）

172.16.129.15/JudgeOnline

【问题描述】

有n个矩形，每个矩形可以用两个整数a、b描述，表示它的长和宽。矩形X(a,b)可以嵌套在矩形Y(c, d)中，当且仅当a < c，b < d，或者b < c，a < d（相当于把矩形X旋转90°）。例如，(1, 5)可以嵌套在(6, 2)内，但不能嵌套在(3, 4)内。

你的任务是选出尽量多的矩形排成一行，使得除了最后一个之外，每一个矩形都可以嵌套在下一个矩形内。如果有多解，矩形编号的字典序应尽量小。

【分析】

矩形之间的“可嵌套” 可以用图来建模。如果矩形X可以嵌套在矩形Y里，就从X到Y连一条有向边。这个有向图是无环的，因为一个矩形无法直接或间接地嵌套在自己内部。换句话说，它是一个DAG（有向无环图）。这样，所要求的便是DAG上的最长路径。

状态：$f[i]$ 表示从i出发能够形成的最长路

状态转移：$f_i = max(0, max_{i,j \in E}(f_j + 1))$

初值：$f[i] = 1$

终值：$max(f[i])$

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=2010;
int x[maxn],y[maxn],n,f[maxn];
int dfs(int u){
    if (f[u]>0) return f[u];
    f[u]=1;
    for (int i=1;i<=n;i++){
        if (x[u]<x[i]&&y[u]<y[i]){
            if (f[u]<dfs(i)+1)
                f[u]=dfs(i)+1;
        }
    }
    return f[u];
}
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d%d",&x[i],&y[i]);
        if (x[i]<y[i]) swap(x[i],y[i]);
    }
    int ans=-1;
    for (int i=1;i<=n;i++){
        dfs(i);
        if (ans<f[i]) ans=f[i];
    }
    printf("%d\n",ans);
}
```

DAG上的DP专练（矩形嵌套1437）

172.16.129.15/JudgeOnline

【问题描述】

在上面基础上，要求输出嵌套方案，多种方案要输出字典序最小的

【分析】

利用状态值，递归寻找最小字典序方案

【参考代码】

```
#include<bits/stdc++.h>
```

```
using namespace std;
const int maxn=2010;
int x[maxn],y[maxn],n,f[maxn];
void pri_ans(int u){
    if (f[u]==1) printf("%d\n",u);else printf("%d ",u);
    for (int i=1;i<=n;i++){
        if (x[u]<x[i]&&y[u]<y[i]&&f[u]==f[i]+1)
            {pri_ans(i);break;
            }
    }
}
int dfs(int u){
    if (f[u]>0) return f[u];
    f[u]=1;
    for (int i=1;i<=n;i++){
        if (x[u]<x[i]&&y[u]<y[i]){
            f[u]=max(f[u],dfs(i)+1);
        }
    }
    return f[u];
}
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d%d",&x[i],&y[i]);
        if (x[i]<y[i]) swap(x[i],y[i]);
    }
    int besti,ans=-1;
    for (int i=1;i<=n;i++){
        dfs(i);
        if (ans<f[i]){
            ans=f[i];
            besti=i;
        }
    }
    printf("%d\n",ans);
    pri_ans(besti);
}
```

DAG上的DP专练（矩形嵌套1567）

172.16.129.15/JudgeOnline

【问题描述】

对于最大嵌套数，输出所有方案，并且按照字典序输出

【分析】

利用状态值寻找方案，并用一个数组记录方案路径上的结点，到边界时输出。

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=2010;
int f[maxn],x[maxn],y[maxn],b[maxn],n;
```

```
int dfs(int u){
    if (f[u]>0) return f[u];
    f[u]=1;
    for (int i=1;i<=n;i++){
        if (x[u]<x[i]&&y[u]<y[i])
            f[u]=max(f[u],dfs(i)+1);
    }
    return f[u];
}

void pri_ans(int u,int step){

    b[step]=u;
    if (f[u]==1) {
        for (int i=1;i<=step-1;i++)
            printf("%d ",b[i]);
        printf("%d\n",b[step]);
        return;
    }
    for (int i=1;i<=n;i++){
        if (x[u]<x[i]&&y[u]<y[i]&&f[u]==f[i]+1)
            pri_ans(i,step+1);
    }
}

int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d%d",&x[i],&y[i]);
        if (x[i]>y[i]) swap(x[i],y[i]);
    }
    int ans=-1;
    for (int i=1;i<=n;i++) {
        dfs(i);
        ans=max(ans,f[i]);
    }
    printf("%d\n",ans);
    for (int i=1;i<=n;i++){
        if (ans==f[i])
            pri_ans(i,1);
    }
}
```

AT_dp_h Grid 1

【题目翻译】

[Grid 1 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]

【解题思路】

二维坐标中的线性DP

走到一个点，只能从它上面或左边走来

状态：$dp[i][j]$ 表示走到（i,j) 坐标时的方案数

转移方程：$dp[i][j]=dp[i-1][j]+dp[i][j-1]$

初值：$dp[1][1]=1$

【参考程序】

```
#include<bits/stdc++.h>
using namespace std;
const int M=1e9+7;
char a[1500][1500];
int f[1500][1500];
int n,m;
int main(){
    scanf("%d%d",&n,&m);
    for (int i=1;i<=n;i++)
        scanf("%s",a[i]+1);
    f[1][1]=1;
    for (int i=1;i<=n;i++){
        for (int j=1;j<=m;j++){
            if (a[i][j]=='#') continue;

            if (i-1>0) f[i][j]=(f[i][j]+f[i-1][j])%M;
            if (j-1>0) f[i][j]=(f[i][j]+f[i][j-1])%M;
        }
    }
    printf("%d\n",f[n][m]);
}
```

AT_dp_q Flowers

【题目翻译】

[Flowers - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]

【解题思路】

线性DP + 线段树优化

状态：$dp[i]$ 表示第i个为止能取得的最大价值

转移：$dp[i]=max(dp[j]+b[i])$ 这里转移需要 $O(n)$ 的时间，总时间复杂度是 $O(n^2)$

利用线段树等数据结构可以将转移优化到 $O(logn)$

【参考程序】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=2e5+50;
long long f[maxn],a[maxn],b[maxn],maxv[maxn*4];
int n;
long long x,y,tmp;
long long query(int o,int L,int R){
    if (x<=L&&R<=y){
        return maxv[o];
    }
    int M=(L+R)>>1;
```

```
    long long ans=0;
    if (x<=M) ans=max(ans,query(o<<1,L,M));
    if (y>M)  ans=max(ans,query(o<<1|1,M+1,R));
    return ans;
}
void update(int o,int L,int R){
    if (L==R){
        maxv[o]=y;
        return;
    }
    int M=(L+R)>>1;
    if (x<=M) update(o<<1,L,M);
    else update(o<<1|1,M+1,R);
    maxv[o]=max(maxv[o<<1],maxv[o<<1|1]);
}
int main(){

    scanf("%d",&n);
    for (int i=1;i<=n;i++) scanf("%lld",&a[i]);
    for (int i=1;i<=n;i++) scanf("%lld",&b[i]);
    long long ans=0;
    for (int i=1;i<=n;i++){
        /*
        f[i]=b[i];
        for (int j=1;j<=i-1;j++)
            if (a[j]<a[i] && f[i]<f[j]+b[i]) f[i]=f[j]+b[i];
        if (i==n) ans=max(ans,f[i]);
        */
        x=1;y=a[i]-1;
        //printf("1111n");
        if (x<=y) tmp=query(1,1,n);else tmp=0;

        f[i]=tmp+b[i];
        x=a[i];y=f[i];
        update(1,1,n);
        ans=max(ans,f[i]);
    }
    printf("%lld\n",ans);


}
```

AT_dp_w Intervals

【题目翻译】

[Intervals - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://)

【解题思路】

线性DP

状态:

$dp[i][j]$ 表示考虑到第 i 个位置，上一个1在j处得最大分数。

状态转移:

$dp[i][i] = min_{j<i}(dp[i-1][j])$

$dp[i][j] = dp[i][j] + \sum_{r[x]=i,l[x]\le j} a[x]$

先做上面的转移，相当于获得 $dp[i][i]$ 的初值

这样的时间复杂度是 $O(n^2)$

然后第二个转移，相当于输入的每一条线段，只要 $r[x] == i$ && $L[x] <= j$ 就能给 所有$dp[i][j]$ 累加值$a[x]$ ，相当于一个区间加

第一个转移，相当于取第一个最大值，线段树统计max，然后调用根结点的max值就行了，再进行单点修改

所以最后时间复杂度可以控制到 $O(nlogn)$

【参考程序】

```
#include<bits/stdc++.h>
#define y1 y3647
#define earse erase
#define LNF 1000000000000000000ll
#define int long long
using namespace std;
inline void read(int &x)
{
    x=0;int f=1;
    char ch=getchar();
    while(ch!=45&&(ch>'9'||ch<'0'))ch=getchar();
    if(ch==45){f=-1,ch=getchar();}
    while(ch<='9'&&ch>='0'){x=x*10+ch-48;ch=getchar();}
    x*=f;
}
const int N=2e5+10;
int i,j,k,n,s,t,m;
struct Segment_Tree{
    int a[N<<2],lazy[N<<2];
    Segment_Tree()
    {
        for(int i=0;i<N<<2;i++)
        a[i]=-LNF;
    }
    void push_up(int rt)
    {
        a[rt]=max(a[rt<<1],a[rt<<1|1]);
    }
    void push_down(int rt)
    {
        if(lazy[rt]==0)return;
        a[rt<<1]+=lazy[rt],a[rt<<1|1]+=lazy[rt];
        lazy[rt<<1]+=lazy[rt],lazy[rt<<1|1]+=lazy[rt];
        lazy[rt]=0;
    }
    void updata(int l,int r,int rt,int x,int y,int c,int op)
    {
        if(x<=l&&y>=r)
        {
            if(op==1)
            a[rt]+=c,lazy[rt]+=c;
```

```
            else
            a[rt]=c;
            return;
        }
        push_down(rt);
        int mid=(l+r)/2;
        if(x<=mid)
        updata(l,mid,rt<<1,x,y,c,op);
        if(y>mid)
        updata(mid+1,r,rt<<1|1,x,y,c,op);
        push_up(rt);
    }
}T;
struct line{
    int l,r,val;
    friend bool operator <(line aa,line bb)
    {
        return aa.r<bb.r;
    }
}a[N];
signed main()
{
    //freopen(".in","r",stdin);
    //freopen(".out","w",stdout);
    //freopen(".ans","w",sdtout);
    read(n),read(m);
    for(i=1;i<=m;i++)
    read(a[i].l),read(a[i].r),read(a[i].val);

    sort(a+1,a+m+1);
    int now=1;
    for(i=1;i<=n;i++)
    {
        int val=max(T.a[1],0ll);
        T.updata(1,n,1,i,i,val,0);
        while(a[now].r==i)
        T.updata(1,n,1,a[now].l,i,a[now].val,1),now++;
    }
    int ans=0;
    ans=max(ans,T.a[1]);
    cout<<ans;
    return 0;
}
```

AT_dp_z Frog 3

【题目翻译】

[Frog 3 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]

【解题思路】

线性DP+斜率优化

【参考程序】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=2e5+50;
```

```
long long dp[maxn];
long long h,t,n,c,a[maxn];
int q[maxn];


double yy(int y){
    return dp[y]+a[y]*a[y];
}

double xl(int y1,int y2){
    return (yy(y1)-yy(y2))/(a[y1]-a[y2]);
}

int main(){
    scanf("%lld%lld",&n,&c);
    for (int i=1;i<=n;i++) scanf("%lld",&a[i]);
    q[1]=1;
    dp[1]=0;
    h=t=1;
    for (int i=2;i<=n;i++){
        while (h<t&&(2*a[i])>=xl(q[h],q[h+1]))h++;
        dp[i]=dp[q[h]]+(a[i]-a[q[h]])*(a[i]-a[q[h]])+c;
        while (h<t&&xl(q[t],i)<=xl(q[t-1],q[t])) t--;
        q[++t]=i;
    }
    printf("%lld\n",dp[n]);


}
```